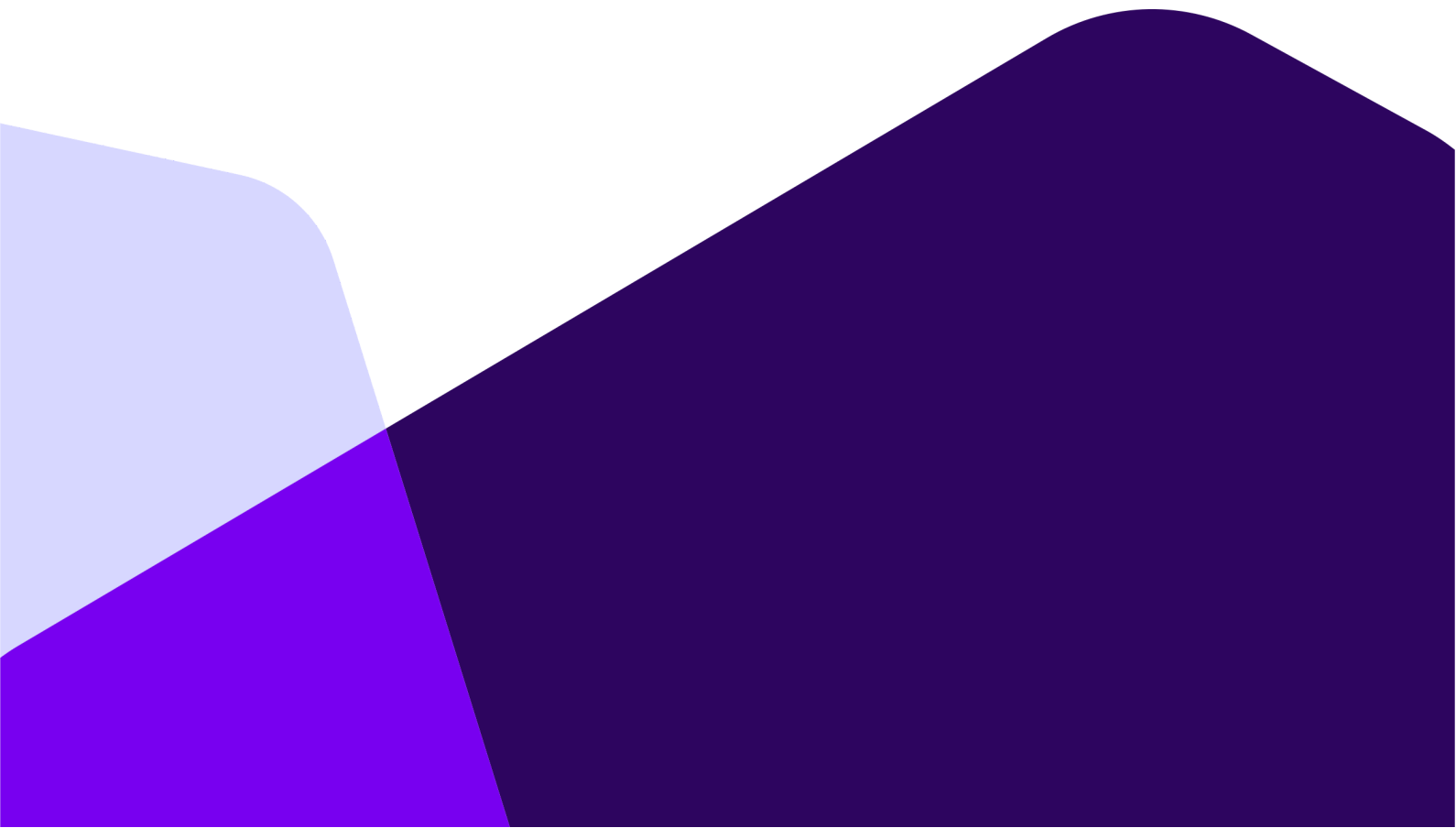


Gruppemedlemmer: Even Unneberg, Sindre Novi, Kristian Thue

Obligatorisk oppgave 1

“”



Innholdsfortegnelse

Oppgave 1	1
Oppgave A. En AnB modell for “Boyd Key Agreement” Protocol.....	1
Oppgave B. Kryptografi for “Boyd Key Agreement” protocol.....	2
Oppgave C. Modifisere SUCI oppgaven fra BTS4410.....	2

Oppgave 1

Oppgave A. En AnB modell for “Boyd Key Agreement” Protocol.

Oppgaven går ut på å modellere Boyd Key Agreement Protocol i AnB notasjon og verifisere sikkerhetsegenskapene ved hjelp av OFMC. Modellen skal inkludere secrecy-mål for nøklene KS og KAB, samt autentiseringsmål baser på MAC-verdier i steg 3 og 4.

Protokollen ble modellert med tre parter: initiator A, responder B og en server s. Serveren oppretter en delt symmetrisk nøkkel, KS, som sendes videre til A og B gjennom billetter beskyttet med langtidsnøkler. Noncene NA og NB brukes for å sikre at meldinger er ferske. Sesjonsnøkkelen KAB er modellert som en nøkkel som avledes fra KS og noncer, og autentisering mellom A og B oppnås ved hjelp av MAC-verdier i de siste stegene av protokollen.

Den ferdige AnB-modellen ble verifisert med OFMC ved bruk av et begrenset antall parallelle sesjoner(--numSess1, --numSess2). Verifikasjonen resulterte i “NO_ATTACK_FOUND”, noe som viser at OFMC ikke fant angrep som bryter de spesifiserte secrecy- eller autentiseringsmålene innenfor denne konfigurasjonen. Protokollen oppfyller kravene til konfidensialitet for KS og KAB, samt autentisering basert på MAC-verdier. Vi prøvde også å kjøre --numSess3, men det førte til svært

lang kjøretid. Output/logfiler ligger i folderen ofmc2026 og heter "BoydKA_numSess1 og BoydKA_numSess2".

Oppgave B. Kryptografi for "Boyd Key Agreement" protocol

For å bruke Boyd Key Agreement Protocol på en sikker måte er det viktig å velge kryptografiske primitiver som gir både konfidensialitet, integritet og autentisering. Protokollen bruker nonces for å sikre ferskhet i nøkkelutvekslingen og for å beskytte mot replay angrep. Noncene blir generert ved å bruke en kryptografisk sikker random generator, vi velger en lengde på 128 bit. Det gir nok sikkerhetsmargin og er ganske vanlig å bruke. Det gir også lav sjanse for kollisjoner.

Protokollen krever også en mekanisme for autentisert kryptering. AES i GCM-mode fungerer til dette. Med nøkkellengde på 256 bit så gir det god sikkerhet og 96 bit nonce følger de anbefalte retningslinjene for GCM mode. Selv om autentisert kryptering allerede gir integritetsbeskyttelse så bruker Boyd-protokollen eksplisitte MAC-verdier i noen steg for å autentisere meldinger mellom partene. For å gjøre dette velger vi HMAC basert på SHA-256. Det er en godt analysert og mye brukt funksjon som gir god beskyttelse.

Sesjonsnøkler avledes fra den etablerte nøkkelen KAB ved hjelp av ConcatKDF basert på SHA-256. ConcatKDF gir en enkel metode for å avlede sesjonsnøkler fra KAB.

Oppgave C. Modifisere SUCI oppgaven fra BTS4410

I denne oppgaven skulle vi skrive om programmene fra SUCI oppgaven. I den originale SUCI løsningen, ble SUCI identifikatoren skrevet til og lest fra fil. Men i denne nye oppgaven skulle programmet i stedet skrives om slik at SUCI utveksles direkte over UDP, der Home og User kjører samtidig og kommuniserer over localhost.

Etter oppdatering av [User.py](#), [Home.py](#) og [SUCI_util.py](#), ble testene gjennomført ved to forskjellige Terminal vinduer. Det ene var i PyCharm, hvor terminalen ble brukt for [User.py](#), mens et vanlig terminalvindu ble brukt for [Home.py](#). Begge programmene ble kjørt lokalt på samme maskin (localhost).

TEST - Home

Først ble et langtids ECDH-nøkkelpar generert ved å kjøre:

```
python ../Home.py keygen
```

Her genererte Home et ECDH P-256 nøkkelpar, og både privat og offentlig nøkkel ble lagret korrekt i PEM-filer:

```
((base) kristianholtethue@Kristian-sin-MacBook-Air TEST_01 % python ../Home.py keygen

Home: Generating long-term ECDH key-pair.
Key-pair generated.
Private key stored in pem-file. Filesize: 412
Public key stored in pem-file. Filesize: 178
Home: Command completed.
```

Deretter ble Home satt i lytte-modus med denne linjen:

```
python ../Home.py deconceal
```

Home knyttet seg til riktig UDP-adresse og port, og ventet på å motta SUCI-datagram. Under testen mottok Home ett UDP-datagram med lengde 340 bytes, som er realistisk gitt innholdet i SUCI (IV, Home-ID-blokk, ephemeral public key i PEM-format, og AES-GCM ciphertext med autentiseringstag):

```
Home: Deconceal command given.
Binding UDP socket on ('127.0.0.1', 5005) with timeout 10s
Waiting for SUCI datagram ...
```

Home skrev mottatt SUCI til fil for debugging, lastet sin private nøkkel, deriverte korrekt session key, verifiserte AAD, og decrypte innholdet til SUCI. Både Home-id og User-id ble korrekt gjenopprettet og logget i terminalen.

```
((base) kristianholtethue@Kristian-sin-MacBook-Air TEST_01 % python ../Home.py deconceal

Home: Deconceal command given.
Binding UDP socket on ('127.0.0.1', 5005) with timeout 5s
Waiting for SUCI datagram ...
Received SUCI datagram from ('127.0.0.1', 58009), Length: 340
SUCI_data written to file 'SUCI_data.bin' (for debugging).
Loaded own private key. Size: 256
Verified Home-ID: 'sidf@home.org'
Recovered User-ID: 'valves strongest warrior'
Sent ACK to ('127.0.0.1', 58009): b'OK'
Home: Command completed.
```

TEST - User

User ble startet i sitt eget terminalvindu med:

```
python ../User.py conceal
```

Her genererte User SUCI på vanlig måte, og sendte denne som et UDP-datagram til Home på port 5005. Lengden på SUCI-dataen var 340 bytes, som samsvarer med

det Home mottok:

```
(base) kristianholtethue@MacBookAir TEST_01 % python ../User.py conceal

User: Concealing a permanent identifier.
      Entity name home: 'sidf@home.org'
      Entity name user: 'valves strongest warrior'
      SUCI_data written to file (debug). Len: 340
      Sending SUCI_data (340 bytes) to HOME at ('127.0.0.1', 5005)
      No ACK received from HOME (timeout).
User: Command completed.
```

I løsningen ble det også implementert en enkel ACK-mekanisme for testingen. Dersom User ikke mottar ACK innen tidsgrensen, loggnes dette som timeout. Timeout-verdien (tiden før timeout logges), kan justeres via SOCKET_TIMEOUT i SUCI_util.py. ACK-mekanismen kan også eventuelt fjernes helt uten at det påvirker funksjonaliteten til SUCI-utvekslingen. Vi gjorde dette hovedsaklig kun for å kunne følge med mer direkte under utviklingen.

Samlet beskrivelse av testen:

Home (deconceal) og User (conceal) ble startet samtidig på localhost i to terminalvinduer. User genererte SUCI og sendte denne som ett UDP-datagram til Home. Home mottok SUCI, deconcealet, og logget Home-id og User-id. Home sendte deretter en enkel "OK"-ACK tilbake til User. Testen ble gjennomført flere ganger uten uforventede feilmeldinger, og bekrefter at SUCI-utveksling over UDP fungerer som forventet.

(da vi gjennomførte testingen, kjørte vi Deconceal rett før Conceal, da vi ikke kunne skrive i to terminalvinduer samtidig. Men dette viste fremdeles at deconceal ventet i listen-mode helt til den mottok SUCI fra User - conceal, eller Timeout slo inn)